

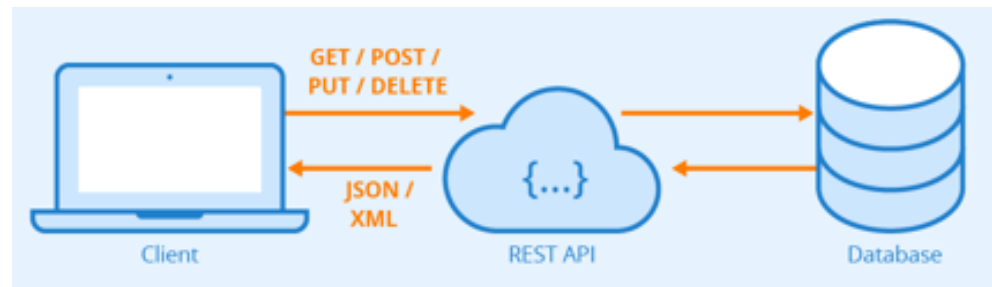
Arquitecturas WEB

Diseño de APIs y Swagger

Documentando una API

¿Qué es una API (Application Programming Interface)?

- Cuando se escribe software, se debe considerar la implementación y la arquitectura del código
- El software es más eficiente cuando se escribe de una forma que la lógica tenga sentido
- En forma adicional, se debe considerar la interacción con los usuarios, así como la interface que se utilizará



Concepto de API

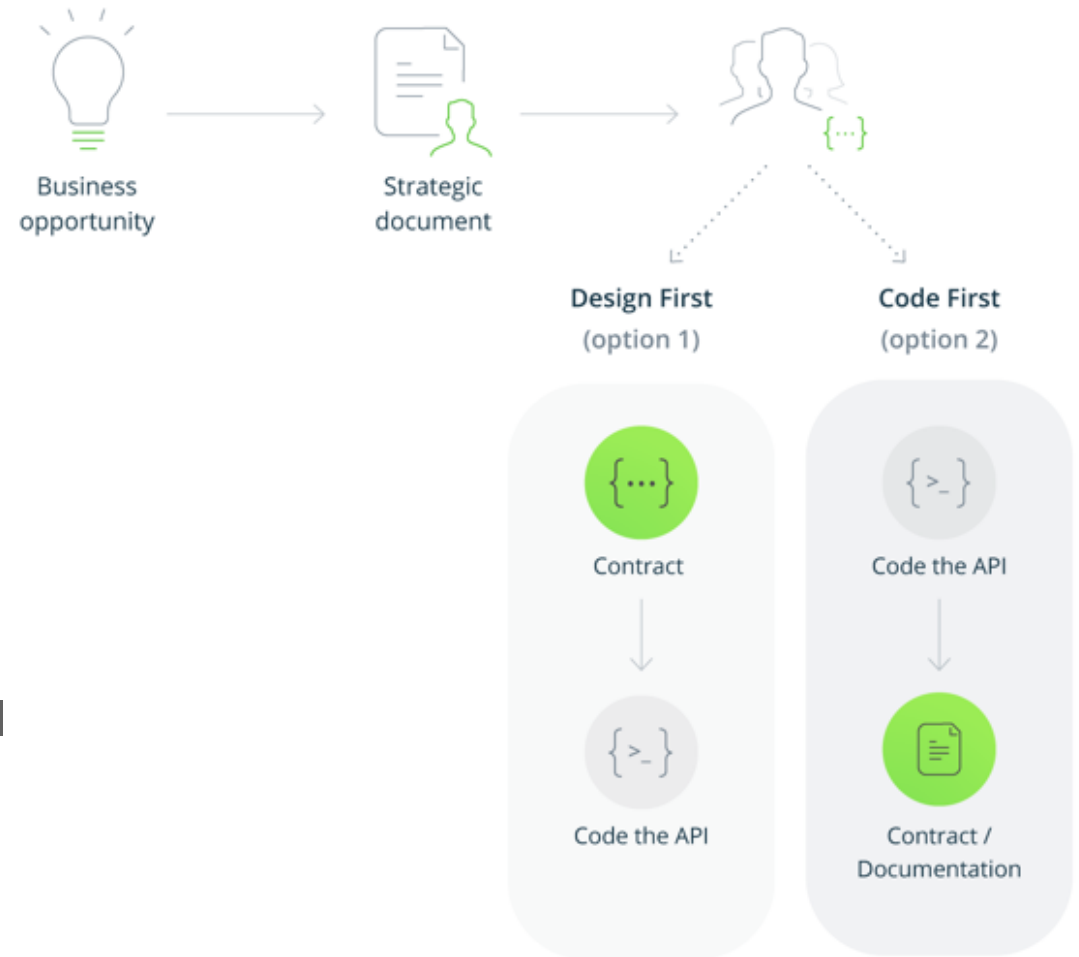
- Un conjunto de definiciones de subrutinas, protocolos de comunicación y herramientas para crear software
 - En términos generales, es un conjunto de métodos de comunicación claramente definidos entre varios componentes
-

Por qué un API?

- Es considerada como un contrato de acciones que se puede solicitar para un servicio en particular
 - Las API se usan hoy en una multitud de aplicaciones web, como redes sociales, software bancario, etc.
 - El contrato estandarizado permite que las aplicaciones externas interactúen con otras
-

Contract-first versus code-first

- Contract-first se refiere a definir y acordar el contrato antes de su implementación. **PREFERIBLE!**
- Code-first se refiere a escribir el código primero, y luego hacer una ingeniería reversa para determinar el contrato



Acciones comunes de un API

- Las APIs son usualmente desarrolladas con el estilo RESTful
- Estas APIs tiene un conjunto de verbos asociados con las acciones del protocolo HTTP, tal como:
 - GET (obtener un único elemento o una colección)
 - POST (agregar un elemento a una colección)
 - PUT (editar un elemento que existe en una colección)
 - DELETE (eliminar un elemento de una colección)

Ventajas de una API

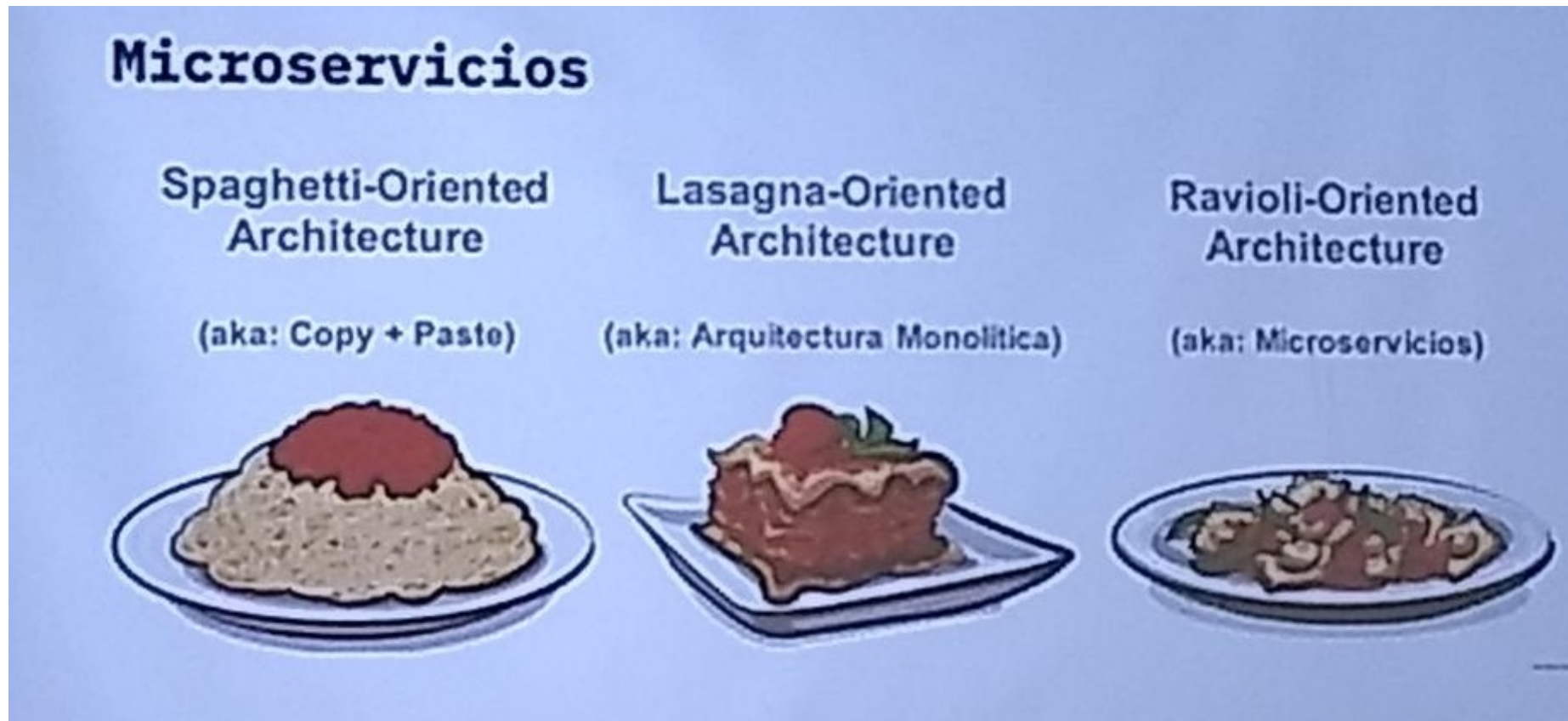
- La ventaja de la coherencia a través de diferentes aplicaciones es tener un estándar al realizar diversas acciones
- Los cuatro verbos de HTTP se correlacionan con las capacidades CRUD que muchas aplicaciones usan hoy en día
- Cuando se trabaja con diferentes APIs en una aplicación, esto constituye una forma reconocible de comprender las implicaciones de las acciones tomadas en diferentes interfaces



¿Qué es un Microservicio?

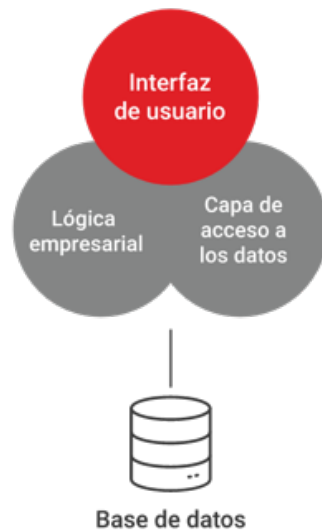
- Es una variante del estilo arquitectónico de arquitectura orientada a servicios (SOA) que estructura una aplicación como una “pequeña” colección de servicios acoplados e independientes de los otros
 - En una arquitectura de microservicios, los servicios son específicos y los protocolos son livianos
-

Arquitectura de Microservicios

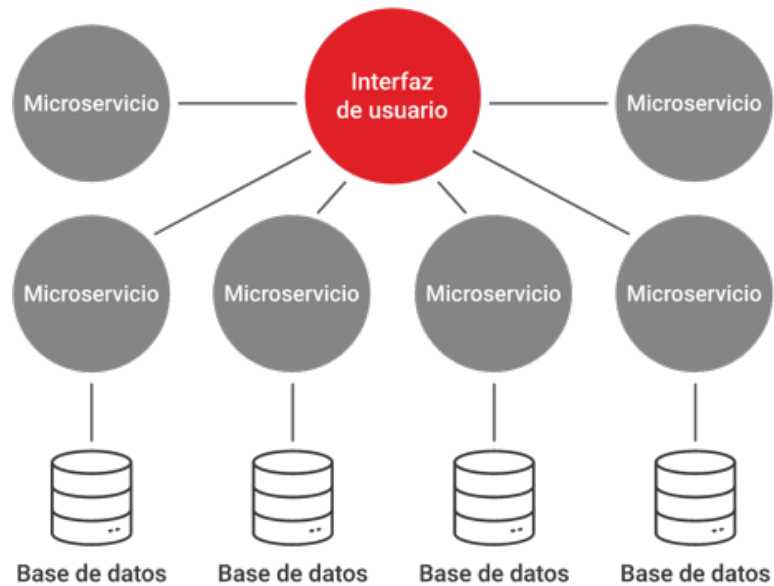


Arquitectura de Microservicios

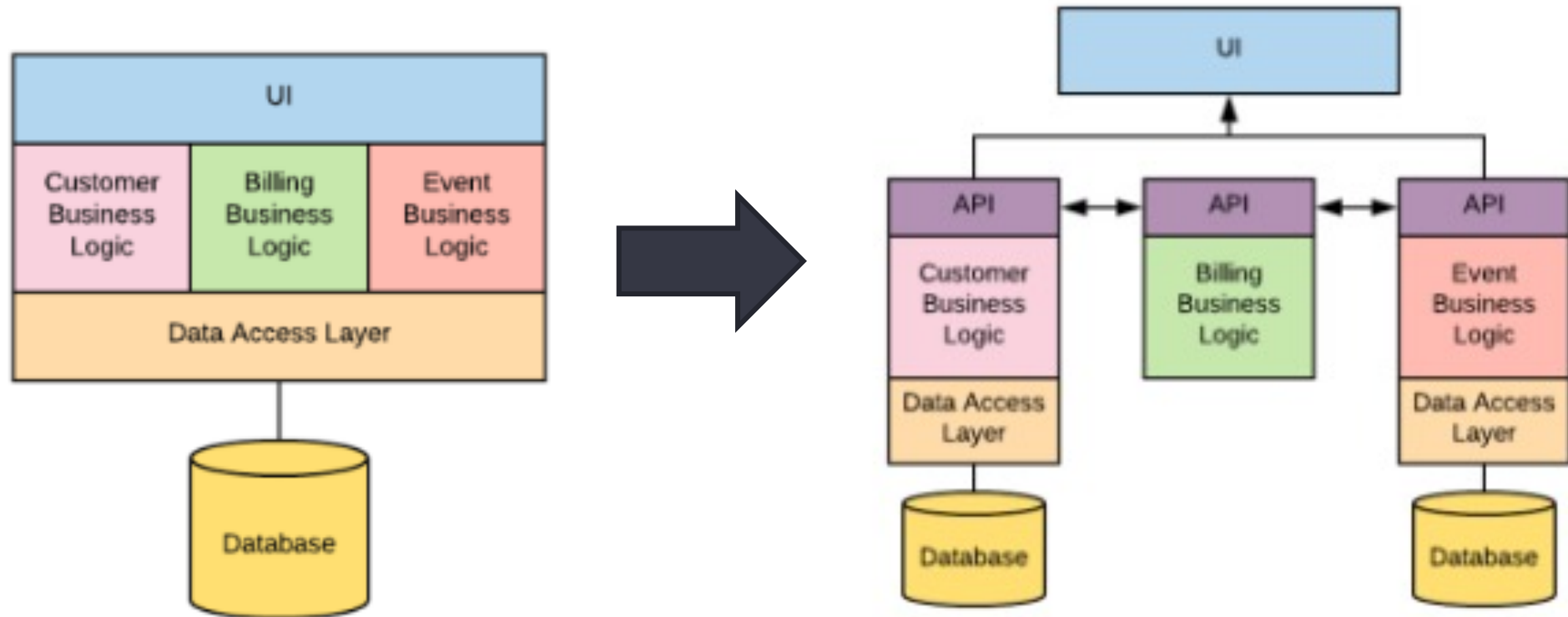
**ARQUITECTURA
MONOLÍTICA**



ARQUITECTURA DE MICROSERVICIOS



Migración de Monolito a Microservicios



Observaciones de la migración

- Las secciones de la lógica empresarial están desglosadas, y cada una de las cuales abarca un microservicio
 - En lugar de tener un límite único para toda la aplicación, la aplicación se divide en pedazos
 - La complejidad de la aplicación se reduce, ya que los diferentes servicios tienen interacciones bien definidos entre sí
 - La capa de interface gráfica de antes solo necesita interactuar con el cliente y los microservicios de eventos, eliminando una dependencia para los microservicios de facturación en la interface gráfica.
 - El microservicio de facturación, por ejemplo, no necesita almacenar datos, por lo que no tiene una capa de acceso a datos, en cambio, interactúa y procesa datos directamente desde el microservicio del cliente y del evento
-

Ventajas de utilizar este tipo de arquitectura

- Estos límites entre las áreas ayudan con el desarrollo (Solo necesitan preocuparse por los microservicios, no por la aplicación completa) y por comprender la arquitectura de la aplicación
 - A diferencia de una aplicación monolítica, un microservicio puede usar una tecnología diferente según sea necesario
 - Las implementaciones de la aplicación en su conjunto se vuelven más enfocadas
 - Los microservicios brindan la flexibilidad de implementar diferentes servicios según sea necesario
-

Diferencia entre APIs y Microservicios

- Un API es un contrato que proporciona orientación para que un consumidor use un determinado servicio
 - Un microservicio es un diseño arquitectónico que separa partes de una aplicación (generalmente monolítica) en pequeños servicios independientes
 - La API sirve como un contrato para las interacciones dentro del microservicio, presentando las opciones disponibles para interactuar con el microservicio
-

Swagger: documentación de APIs

- Las APIs no comparten un idioma común en demasiadas ocasiones y Swagger y Swagger UI trabajan para solucionarlo
 - Las APIs nos permiten conectar y compartir datos
 - Una API pierde su sentido sino es accesible y si no tenemos una documentación que nos ayude a entenderla
-

Swagger: documentación de APIs

- Uno de los mayores problemas de las APIs es que en muchos casos, la documentación que les acompaña es inútil
 - Swagger nace con la intención de solucionar este problema
 - Su objetivo es estandarizar el vocabulario que utilizan las APIs
 - Es el diccionario API
 - Swagger es una serie de reglas, especificaciones y herramientas que nos ayudan a documentar nuestras APIs
-

Ventajas de Swagger



- Todo el mundo entiende Swagger
- Es fácilmente procesable por una computadora
- Con Swagger la documentación puede utilizarse directamente para automatizar procesos dependientes de APIs
- SwaggerHub es una tool para automatizar el desarrollo de APIs

Documento Swagger

- Swagger utiliza un documento JSON o YAML (YAML Ain't Markup Language) existente y lo hace interactivo
 - Crea una plataforma que ordena cada uno de los métodos (get, put, post, delete) y categoriza las operaciones
 - Cada uno de los métodos es expandible, y en ellos podemos encontrar un listado completo de los parámetros con sus respectivos ejemplos
-

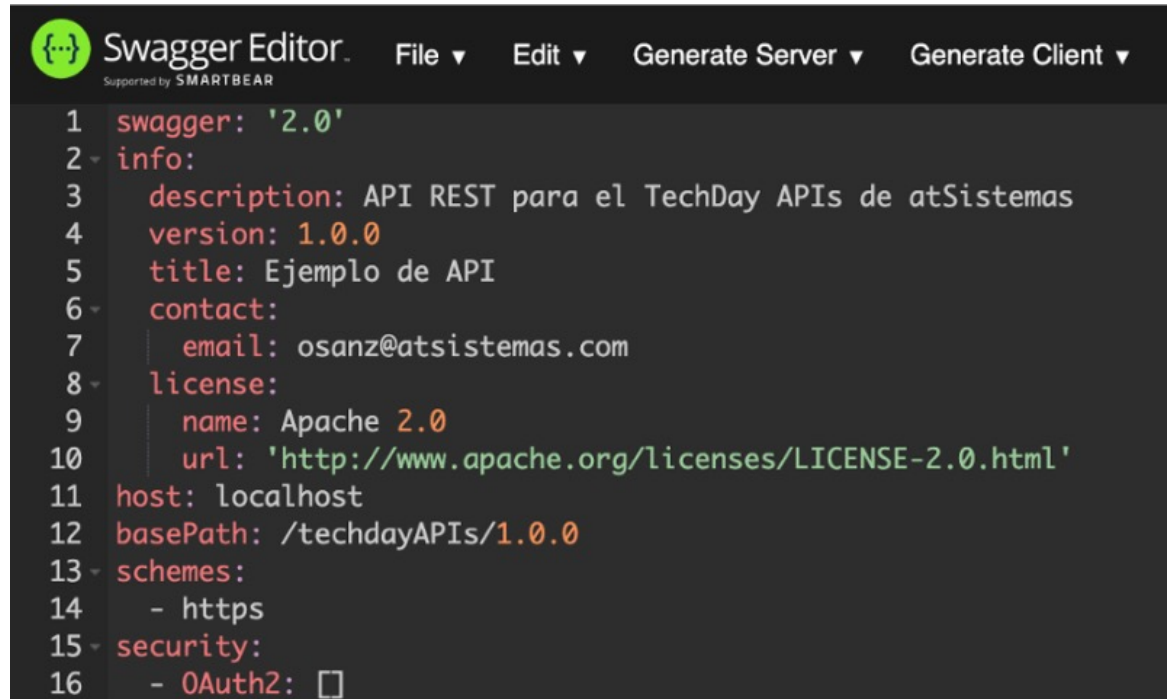
Herramientas para documentar con Swagger

- SwaggerHub
- Swagger UI
- Theneo



Documento Swagger

Una API bien documentada
significa que es accesible por otros
desarrolladores, y haciendo las
APIs más accesibles pueden ser
mejorarlas

The image shows the Swagger Editor web application. The interface has a dark theme. At the top, there is a header with the Swagger Editor logo (a green circle with a curly brace), the text "Swagger Editor", and a subtext "Supported by SMARTBEAR". To the right of the header are four menu items: "File", "Edit", "Generate Server", and "Generate Client", each with a dropdown arrow. The main area is a code editor with line numbers on the left (1 to 16). The code is a JSON object representing an API configuration. The code is as follows:

```
1  swagger: '2.0'
2  info:
3    description: API REST para el TechDay APIs de atSistemas
4    version: 1.0.0
5    title: Ejemplo de API
6    contact:
7      email: osanz@atsistemas.com
8    license:
9      name: Apache 2.0
10     url: 'http://www.apache.org/licenses/LICENSE-2.0.html'
11  host: localhost
12  basePath: /techdayAPIs/1.0.0
13  schemes:
14    - https
15  security:
16    - OAuth2: []
```

Swagger Hub

<https://swagger.io/>

The screenshot displays the Swagger Hub interface for a Swagger 2.0 API. The top header features the SMARTBEAR SwaggerHub logo. Below the header, a breadcrumb trail shows '← Prueba 1.0.0'. The left sidebar contains navigation links for 'Info', 'Tags', 'Search', and a list of API endpoints under the 'pet' resource. The main content area displays the Swagger 2.0 JSON definition, which includes metadata like version (1.0.0), title (Swagger Petstore), and contact information, as well as a list of API endpoints with their respective methods and descriptions.

SMARTBEAR
SwaggerHub™

← Prueba 1.0.0

Info

Tags

Search

pet ^

- POST /pet
- PUT /pet
- GET /pet/findByStatus
- GET /pet/findByTags
- GET /pet/{petId}
- POST /pet/{petId}
- DELETE /pet/{petId}
- POST /pet/{petId}/uploadImage

1 swagger: '2.0'

2 info:

3 description: |

4 This is a sample Petstore server. You can find

5 out more about Swagger at

6 http://swagger.io or on

7 [irc.freenode.net, #swagger](http://swagger.io/irc/).

8 version: 1.0.0

9 title: Swagger Petstore

10 termsOfService: http://swagger.io/terms/

11 contact:

12 email: apiteam@swagger.io

13 license:

14 name: Apache 2.0

15 url: http://www.apache.org/licenses/LICENSE-2.0.html

16 # host: petstore.swagger.io

17 # basePath: /v2

18 tags:

19 - name: pet

20 description: Everything about your Pets

21 externalDocs:

22 description: Find out more

23 url: http://swagger.io

24 - name: store

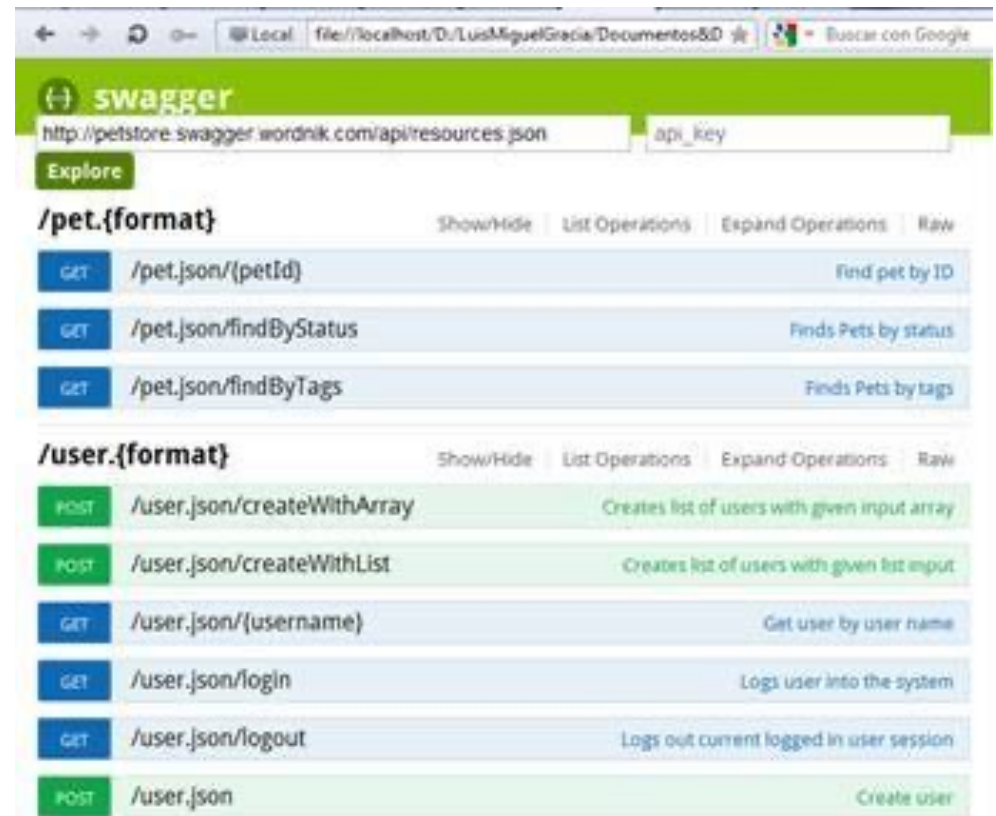
25 description: Access to Petstore orders

26 - name: user

27 description: Operations about user

Uso de Swagger en SpringBoot

- Permite anotar código y luego generar automáticamente la documentación
- Luego, se pueden explorar visualmente los elementos del API REST, e incluso probar los endpoints



Uso de Swagger en SpringBoot

```
@RestController
@Tag(name = "Published memes", description = "Published memes related resources")
@RequestMapping(produces = MediaType.APPLICATION_JSON_VALUE)
public class PublishedMemesController {

    new *

    @GetMapping("/top/{stream}/last/30d")
    @Operation(summary = "Get the top memes since last 30 days for a stream",
        description = "Top meme's list (page) since 30 last days for a stream")
    public List<PublishedMeme> getTop30d(
        @Parameter(description = "Stream name", example = "gaming") @PathVariable String stream,
        @Parameter(description = "Page number") @RequestParam(value = "page", default = "1") Integer page) {
    }
```

OpenAPI

- Es una evolución de Swagger (2016)
- La adopción masiva de OpenAPI permitiría mejoras considerables en la interconectividad entre APIs y una menor necesidad de adaptaciones



Swagger vs OpenAPI

- Aunque los términos alguna vez se refirieron a lo mismo, ya no se pueden usar indistintamente... aunque algunas personas todavía lo hacen
- En 2021, OpenAPI se refiere a la especificación estándar de la industria para el diseño de API RESTful
- Swagger se refiere a un conjunto de herramientas de SmartBear

OpenAPI = La especificación en sí, anteriormente conocida como especificación

Swagger = Herramientas utilizadas en la implementación de OpenAPI

Arquitecturas WEB

Swagger